Overlap 的实现方法和背后的逻辑

> **原始问题：**1. overlap 的实现方法和背后的逻辑

提问：2026/07/23 22:07　更新：2026/07/27 01:53　重要度：5/5

一句话结论

Overlap 的本质是：让 GPU 在等待某件慢事情（通信、数据搬运、CPU 下发指令）完成时不要闲着，而是同时去做另一件不依赖该结果的事情，从而用并行把等待时间“藏”起来。

它的实现依赖三个底层能力：异步执行 + 多条独立的执行流(stream/engine) + 依赖关系管理。理想情况下总时间 ≈ max(计算, 通信)，而不是二者相加。

直觉：餐厅上菜模型

把 GPU 想象成厨师，把通信/拷贝想象成“跑堂端菜到隔壁桌”。

- 没有 overlap（串行）：厨师炒完一道菜 → 亲自端出去 → 回来再炒下一道。端菜时厨师完全空闲。
- 有 overlap：厨师炒完一道菜交给跑堂去端（异步），自己立刻开始炒下一道。端菜和炒菜同时进行。

关键前提：炒下一道菜不依赖“上一道菜已经端到桌上”。如果有依赖就无法重叠。类比失效点：现实中跑堂(通信引擎)和厨师(计算SM)可能抢同一资源(如显存带宽、SM)，所以 overlap 不是完全免费的并行。

底层逻辑：为什么 GPU 能 overlap

1) GPU 执行是异步的：CPU 调用 kernel 后不等它执行完，把任务提交到队列就立即返回，GPU 在后台执行。下发与执行解耦，CPU 才有机会提交别的任务。

2) GPU 有多个独立执行引擎：Compute Engine(SM) 执行计算 kernel；Copy Engine(DMA) 专门做显存拷贝(H2D/D2H)，独立于 SM；NCCL 通信通常也是 kernel，占用 SM 或专用路径。Copy Engine 与 Compute Engine 物理独立，所以“拷贝”和“计算”天然可真并行。

3) CUDA Stream：表达“哪些任务可以并行”。同一条 stream 内 FIFO 串行；不同 stream 之间默认无顺序约束，可自由并行。所以 overlap 的实现套路就是：把两件想并行的事放进两条不同 stream，用 event 管理跨流依赖。

关键图：串行 vs 重叠的时间线



公式与量级

```
无 overlap（串行）：  T_total = T_comp + T_comm
理想 overlap:        T_total ≈ max(T_comp, T_comm)

例：T_comp = 100 ms, T_comm = 40 ms
  串行   = 100 + 40 = 140 ms
  重叠   = max(100, 40) = 100 ms   → 加速 1.4×，省约 28.6%

例：T_comm = 120 ms（通信占比更大）
  串行   = 100 + 120 = 220 ms
  重叠   = max(100, 120) = 120 ms  → 加速 1.83×

物理下界：T_total ≥ max(T_comp, T_comm)，永远不可能更低。
```

最小 Sample：拷贝-计算 overlap

```
import torch
N_CHUNKS = 4
copy_stream = torch.cuda.Stream()   # 一条流负责拷贝
comp_stream = torch.cuda.Stream()   # 一条流负责计算

# pinned 内存是 overlap 成立的隐藏前提，否则拷贝退化成同步
host = [torch.randn(1024,1024).pin_memory() for _ in range(N_CHUNKS)]
dev  = [torch.empty(1024,1024, device='cuda') for _ in range(N_CHUNKS)]
out  = [None]*N_CHUNKS

for i in range(N_CHUNKS):
    with torch.cuda.stream(copy_stream):
        dev[i].copy_(host[i], non_blocking=True)   # 异步拷贝第 i 块
    comp_stream.wait_stream(copy_stream)           # 跨流依赖
    with torch.cuda.stream(comp_stream):
        out[i] = dev[i] @ dev[i]                   # 计算第 i 块
torch.cuda.synchronize()

# 效果：算第 i 块时，第 i+1 块的拷贝同时进行
# 要点：overlap = 拆分 + 多流 + 依赖管理
```

常见误区与失败模式

1) 误以为多流一定并行：若两条流都要用 SM 而 SM 已被占满，通信 kernel 抢不到资源，overlap 名存实亡（NCCL 占 SM 是典型）。
2) 忘记 pinned memory：拷贝退化成同步，overlap 失效，而且不报错只是变慢，极难发现。
3) 依赖没管好：跨流没用 event 同步，计算读到没传完的数据，导致数值错误或 NaN。
4) bucket 太小：每次通信太小，启动开销占比过高反而变慢；太大又减少重叠机会。
5) 只看平均不看时间线：必须用 Nsight Systems / torch.profiler 确认计算块和通信块真的在时间上重叠。